```
ditlog
direction TB
REPOINVCHANGELOGUSERSONBOARDING[("auditlog/users/<br />onboarding.yml/json/csv")]
REPOINVCHANGELOGUSERSOFFBOARDING[("auditlog/users/<br />offboarding.yml/json/csv")]
REPOINVCHANGELOGUSERSCHANGES[("auditlog/users/<br />attributes.yml/json/csv")]
REPOINVCHANGELOGATTRIBUTE[("auditlog/attribute/<br />{attribute}.yml/json/csv")]
REPOINVCHANGELOGROLE[("auditlog/role/<br />{role}.yml/json/csv")]
REPOINVCHANGELOGOU[("auditlog/ou/<br />{ou}.yml/json/csv")]
end
end
end

subgraph accessctl GitLab CI/CD Pipeline Jobs
direction LR
subgraph Manifest Stage
direction LR
CIMANIFESTUSERJOB[["Stage 1.1<br />Users Job<br/>CLI manifest:users"]]
CIMANIFESTROLEJOB[["Stage 1.2<br />Roles Job<br />CLI manifest:roles"]]
CIMANIFESTGROUPJOB[["Stage 1.3<br />Org Units Job<br/>CLI manifest:ou"]]
end
subgraph Auditlog Stage
direction LR
CICHANGELOGUSERJOB[["Stage 2.1<br />Users Job<br/>CLI auditlog:users"]]
CICHANGELOGATTRIBUTEJOB[["Stage 2.2<br />Attributes Job<br/>CLI auditlog:attributes"]]
CICHANGELOGROLEJOB[["Stage 2.3<br />Roles Job<br/>CLI auditlog:roles"]]
CICHANGELOGGROUPJOB[["Stage 2.4<br />Org Units Job<br/>CLI auditlog:ou"]]
end
subgraph Provisioning Stage
direction LR
subgraph Okta API
CIPROVISIONINGOKTAUSERJOB["Stage 3.1<br />Okta Users Job<br />provision:okta-users"]:::orange
CIPROVISIONINGOKTAGROUPJOB["Stage 3.2<br />Okta Groups Job<br />provision:okta-
groups"]:::orange
end
subgraph Google Workspace Directory API
CIPROVISIONINGGOOGLEGROUPJOB["Stage 3.3<br />Google Groups Job<br />provision:google-groups
end
subgraph GitLab.com SaaS API
CIPROVISIONINGGITLABSAASGROUPJOB["Stage 3.4<br />GitLab SaaS Groups Job<br />provision:gitla
saas-groups"]
end
subgraph GitLab Self-Managed Instance API
CIPROVISIONINGGITLABSELFGROUPJOB["Stage 3.5<br />GitLab Self-Managed Groups Job<br
/>provision:gitlab-self-groups"]
end
end

CIMANIFESTUSERJOB --> CIMANIFESTROLEJOB --> CIMANIFESTGROUPJOB
CICHANGELOGUSERJOB --> CICHANGELOGATTRIBUTEJOB --> CICHANGELOGROLEJOB --> CICHA
CIPROVISIONINGOKTAUSERJOB --> CIPROVISIONINGOKTAGROUPJOB --> CIPROVISIONINGGOOG
```

```
CIPROVISIONINGGITLABSAASGROUPJOB --> CIPROVISIONINGGITLABSELFGROUPJOB

click CIMANIFESTUSERJOB "/handbook/security/identity/platform/manifests" "View Details"
click CIMANIFESTROLEJOB "/handbook/security/identity/platform/manifests" "View Details"
click CIMANIFESTGROUPJOB "/handbook/security/identity/platform/manifests" "View Details"
click CICHANGELOGUSERJOB "/handbook/security/identity/platform/auditlog" "View Details"
click CICHANGELOGATTRIBUTEJOB "/handbook/security/identity/platform/auditlog" "View Details"
click CICHANGELOGROLEJOB "/handbook/security/identity/platform/auditlog" "View Details"
click CICHANGELOGGROUPJOB "/handbook/security/identity/platform/auditlog" "View Details"
click CIPROVISIONINGOKTAUSERJOB "/handbook/security/identity/platform/provisioning/okta" "View
Details"
click CIPROVISIONINGOKTAGROUPJOB "/handbook/security/identity/platform/provisioning/okta" "View
Details"
click CIPROVISIONINGGOOGLEGROUPJOB "/handbook/security/identity/platform/provisioning/google"
"View Details"
click CIPROVISIONINGGITLABSAASGROUPJOB
"/handbook/security/identity/platform/provisioning/gitlab" "View Details"
click CIPROVISIONINGGITLABSELFGROUPJOB
"/handbook/security/identity/platform/provisioning/gitlab" "View Details"

classDef slate fill:cbd5e1,stroke:475569,stroke-width:1px;
classDef red fill:fca5a5,stroke:dc2626,stroke-width:1px;
classDef orange fill:fdba74,stroke:ea580c,stroke-width:1px;
classDef yellow fill:fcd34d,stroke:ca8a04,stroke-width:1px;
classDef emerald fill:6ee7b7,stroke:059669,stroke-width:1px;
classDef cyan fill:67e8f9,stroke:0891b2,stroke-width:1px;
classDef sky fill:7dd3fc,stroke:0284c7,stroke-width:1px;
classDef violet fill:c4b5fd,stroke:7c3aed,stroke-width:1px;
classDef fuchsia fill:f0abfc,stroke:c026d3,stroke-width:1px;
end


 CI/CD Job Workflows for Okta

mermaid
graph TB

subgraph Identity GitLab Repositories
subgraph accessctl-inventory Repo
direction TB
REPOINVMANIFESTSUSERS[("manifests/users/<br />users.yml/json/csv")]:::sky
REPOINVMANIFESTSROLES[("manifests/roles/<br />{role}.yml/json/csv")]:::sky
REPOINVMANIFESTSOU[("manifests/ou/<br />{ou}.yml/json/csv")]:::sky
end
end

subgraph Identity Platform CI/CD Provisioning State Scripts
direction LR

CIUSERJOB["Stage 3.1<br />Okta Users Job<br />provision:okta-users"]:::orange
```

```
CIGROUPJOB["Stage 3.2<br />Okta Groups Job<br />provision:okta-groups"]:::orange

CIUSERPARSEAPI[(Get All Users<br/>from Okta API)]
CIUSERCHECKROLE{{Compare user manifest<br />with API results to check for<br />rbacrole differences}}
CIUSERUPDATEAPI[[Update Okta users with<br />updated rbacrole attribute]]:::violet
CIUSERJOB --> CIUSERPARSEAPI
CIUSERPARSEAPI --> CIUSERCHECKROLE
CIUSERCHECKROLE --> CIUSERUPDATEAPI

CIGROUPJOB --> CIGROUPPARSEMANIFEST
CIGROUPPARSEMANIFEST[(Parse Manifest)]
CIGROUPPARSEAPI[(Get rbac groups from Okta API)]

CIGROUPCHECKEXISTS{{Check if group exists}}
CIGROUPCREATEGROUP[[Create group]]:::violet
CIGROUPDELETEGROUP[[Delete group]]:::violet
CIGROUPCHECKUSERS{{Check if manifest users exist in Okta Group Users API}}
CIGROUPCREATEUSER[[Attach user to group<br />if does not exist]]:::violet
CIGROUPDELETEUSER[[Detach user from group<br />if no longer in policy]]:::violet
CIGROUPLOGS3{{Create Audit Log entry in S3 bucket}}:::red
CIGROUPAUDIT{{Audit Transaction<br />REST API Call to accessctl<br />for automation workflows}}:::red
end


subgraph Identity SaaS Vendor Services
direction LR
subgraph Okta GitLab Tenant
IDENTITYVENDOROKTAAPIUSERS[("Okta Users")]
IDENTITYVENDOROKTAAPIGROUPS[("Okta Groups")]
IDENTITYVENDOROKTAAPIGROUPUSERS[("Okta Group Users")]
IDENTITYVENDOROKTAAPIENDPOINT[("Okta API<br />https://gitlab.okta.com/api/v1")]
IDENTITYVENDOROKTAAPIENDPOINT --> IDENTITYVENDOROKTAAPIUSERS
IDENTITYVENDOROKTAAPIENDPOINT --> IDENTITYVENDOROKTAAPIGROUPUSERS
IDENTITYVENDOROKTAAPIENDPOINT --> IDENTITYVENDOROKTAAPIGROUPS
IDENTITYVENDOROKTAAPIUSERS -.- IDENTITYVENDOROKTAAPIGROUPUSERS
IDENTITYVENDOROKTAAPIGROUPS -.- IDENTITYVENDOROKTAAPIGROUPUSERS
end
end

CIGROUPPARSEMANIFEST --> CIGROUPPARSEAPI
CIGROUPPARSEAPI --> CIGROUPCHECKEXISTS
CIGROUPCHECKEXISTS --> CIGROUPCREATEGROUP
CIGROUPCREATEGROUP ---> IDENTITYVENDOROKTAAPIENDPOINT
CIGROUPCHECKEXISTS --> CIGROUPCHECKUSERS
CIGROUPCHECKEXISTS --> CIGROUPDELETEGROUP
CIGROUPCREATEGROUP --> CIGROUPCHECKUSERS
CIGROUPDELETEGROUP --> CIGROUPCHECKUSERS
CIGROUPCREATEUSER ----> IDENTITYVENDOROKTAAPIENDPOINT
```

CIGROUPCHECKUSERS --> CIGROUPCREATEUSER
CIGROUPCHECKUSERS --> CIGROUPDELETEUSER
CIGROUPCREATEUSER --> CIGROUPLOGS3
CIGROUPDELETEUSER --> CIGROUPLOGS3
CIGROUPLOGS3 --> CIGROUPAUDIT
CIUSERUPDATEAPI ----> IDENTITYVENDOROKTAAPIENDPOINT
CIGROUPDELETEGROUP ---> IDENTITYVENDOROKTAAPIENDPOINT
CIGROUPDELETEUSER ---> IDENTITYVENDOROKTAAPIENDPOINT
REPOINVMANIFESTSUSERS --> CIUSERJOB
REPOINVMANIFESTSROLES --> CIGROUPJOB
REPOINVMANIFESTSOU --> CIGROUPJOB

classDef slate fill:cbd5e1,stroke:475569,stroke-width:1px;
classDef red fill:fca5a5,stroke:dc2626,stroke-width:1px;
classDef orange fill:fdba74,stroke:ea580c,stroke-width:1px;
classDef yellow fill:fcd34d,stroke:ca8a04,stroke-width:1px;
classDef emerald fill:6ee7b7,stroke:059669,stroke-width:1px;
classDef cyan fill:67e8f9,stroke:0891b2,stroke-width:1px;
classDef sky fill:7dd3fc,stroke:0284c7,stroke-width:1px;
classDef violet fill:c4b5fd,stroke:7c3aed,stroke-width:1px;
classDef fuchsia fill:f0abfc,stroke:c026d3,stroke-width:1px;

SECTION: security/planning/_index.md

Security Planning

The Security Planning page catalogs the planning work prior to implementation offSecurity Department initiatives and projects that span multiple security teams and projects, or across the entire GitLab organization. This includes things from high level architecture designs for tools used by the Security to developing new or maturing organizational processes.

A Security Plan is done when the text is sufficient to create Epics, Milestones, and Issues with a enough detail to begin work. Requiring further work in the form of proof-of-concepts or spikes may also be an outcome of a "complete" Plan.

Security Plans are not meant to be the single-source-of-truth (SSOT) once implementation begins. Non-planning handbook pages, runbooks, and system documentation in related source repositories should be kept up-to-date and continue to be iterated on.

It is suggested to associate a Security Plan with a single Epic and maintain a link to that Epic in this page. All created issues can be associated with that Epic as an alternative to providing links in the Security Plan

Workflow

Security Plans begin with an Epic or other action item that does not have
enough detail to begin an iteration or develop on MVC:

1. If source is an issue, promote it to an Epic.
1. Create a new Security Plan in this directory and link to the Epic.
1. Use MR comments to draw particular attention to any areas you would like
   specific feedback.
1. Issues can start being created from the Plan at any time.
1. Once the MR is merged, create issues for any remaining work that is not
   covered.

Tips

- Use headers () or manually create anchors ({: name="hello-world"})
  to make it easier to refer to particular section of the Plan from GitLab.

Plans

Plan	Other Resources and Links
Security Requirements for Development and Deployment	

SECTION: security/planning/security-development-deployment-requirements.md

Summary

The Security Department and its subteams are responsible for a number of
applications and other resources that process various types of sensitive data
in service of supporting GitLab.com.

Scope

The scope of the requirements and practices documented in this page are Security
Department tools and resources that collect, process, and store RED data.

What's in this page

- Infrastructure requirements for security team tools that collect, process, and
  and store RED data.

What is not in this page

- An exhaustive list of security best practices for network, host, and infrastructure security.
- An exhaustive list of security best practices for application, database,
  cloud function, or other development resources.
- Prescriptive guidelines for development and change management.

Review Process

Future work: At this time, we don't have defined process for architecture and implementation review like we have for AppSec Reviews.
Such a process will be the responsibility of a future Security team.

Requirements

Overall Guidelines

The following requirements are driven by 3 high level guidelines:

- Least Privilege
- Zero Trust
- GitLab's Security Controls

Identity, Authentication, and Authorization

Accounts and Credentials

1. Shared user accounts MUST NOT be used.
1. Multi-factor (MFA) authentication MUST be enabled for all user accounts.
1. Access to service account credentials MUST follow the access control of the resources to which the credentials grant access.
1. Service account credentials MUST be rotated every 365 days.
1. Non-MFA user account credentials (for example, API keys) MUST be rotated every 90 days.

Identity

1. An identity provider MUST be used for all external accounts.

Authentication

1. Service accounts MAY be authenticated using static credentials, such as API tokens or shared private keys.

Authorization

1. User access to sensitive data MUST be granted through security groups.
1. Individual access for disaster recovery MAY be granted to system owners as specified in the tech stack tracking file.

Service Account usage

1. Service accounts names SHOULD be meaningful.
1. Service accounts with access to RED data MUST follow the Access Request process.
1. Service accounts with access to RED data MUST be limited to single logical scope; for example, a single GCP project.

Network Security

1. All network firewalls MUST be configured such that the default policy is DENY.
1. Network firewall rules SHOULD deny egress by default.
1. All external communication MUST be encrypted in transit using up to date protocals and ciphers.
1. All internal communication SHOULD be encrypted in transit if possible.

Data Handling and Isolation

1. Data retention policies MUST be followed.
1. Data MUST be encrypted at rest.
    1. Data MAY be encrypted using provider managed keys.
1. Data of different types MUST be logically seperated at rest.
1. Virtual networks (for example, VPC in GCP) MAY be used as a mechanism for data and workload isolation.

Examples of different data types:

- User content, such as repository contents or attachments
- Production derived data, such as logs
- DFIR (Digital Forensics and Incident Reponse) artifacts, such as system logs and disk images

Vulnerability and Patch Management

1. Resources MUST be covered by the Security Vulnerability Management process.

Change Management and Tracking

1. Changes to systems that process RED data MUST be tracked in a corresponding issue, merge request, or other reviewable process.

Audit Logging

1. Environment audit logs MUST be enabled and stored in accordance with retention policies.
1. Application audit logs, if supported and available, MUST be enabled and stored in accordance with retention policies.
1. Logs MUST be forwarded and processed in a centralized location that provides access to any operational team, such as Security Operations.

Best Practices

The following practices are meant to provide more specific technical implementation examples to meet the above criteria.

GCP Practices

Disable legacy metadata endpoints

The legacy metadata endpoints should be disabled on all projects.

Multiple Environments

Development and deployment should occur in a minimum of 2 environments, in addition to local development:

- A shared testing, integration, or other non-production environment.
- A production environment which meets or exceeds the requirements in this page.

The environments should be configured as closely as possible as a best practice to reduce errors in deployment due to inconsistent configuration.

GCP Security Group (gcp--sg@gitlab.com) Conventions

Security groups for gitlab.com organization resources should be named gcp-:grouppurpose-sg@gitlab.com.

Remove default service account permissions

When services are enabled, the default service account may be created with project/editor permissions.

- This default mapping should be removed.
- A service account for each workload should be provisioned with the least privileges required.

Use Lifecycle Management for GCS

Buckets which store data subject to retention requirements should use lifecycle management to automate deletion.

terraform
```
resource "googlestoragebucket" "system-logging-archive" {
  name          = "${var.projectid}-system-logging-archive"
  project       = var.projectid
  location      = "US"
  forcedestroy = true

  lifecyclerule {
    condition {
      age = "365"
    }
    action {
      type = "Delete"
    }
  }
}
```

Shielded VMS

Shielded VMs should be used in all cases. This can be enforced with organization or folder policy. Since this can't be enforced on the existing

gitlab.com organization due to existing workloads, it should enforced at the folder or project level for new folders and projects.

terraform
```
resource "googlefolderorganizationpolicy" "shieldedvmpolicy" {
  folder     = "myfolder"
  constraint = "compute.requireShieldedVm"

  booleanpolicy {
    enforced = true
  }
}
```

Enable shielded VMs for k8s node pools
```
resource "googlecontainernodepool" "nodes" {
  ...

  enableshieldednodes = true
  nodeconfig {
    ...

    shieldedinstanceconfig {
      enablesecureboot          = true
      enablevtpm                = true
      enableintegritymonitoring = true
    }

    metadata = {
      disable-legacy-endpoints = "true"
    }
  }

  lifecycle {
    ignorechanges = [
      nodeconfig,
    ]
  }
}
```

VPC Firewall Rules

A default DENY rule for both egress and ingress should be added to all VPCs:

terraform
```
resource "googlecomputenetwork" "vpc" {
   ...
}

resource "googlecomputefirewall" "deny-all-egress" {
```

```
  name      = "${googlecomputenetwork.vpc.name}-deny-all-egress"
  project   = var.projectid
  network   = googlecomputenetwork.vpc.selflink
  priority  = 65533
  direction = "EGRESS"

  deny {
    protocol = "all"
  }
}

resource "googlecomputefirewall" "deny-all-ingress" {
  name      = "${googlecomputenetwork.vpc.name}-deny-all-ingress"
  project   = var.projectid
  network   = googlecomputenetwork.vpc.selflink
  priority  = 65533
  direction = "INGRESS"

  deny {
    protocol = "all"
  }
}
```

Least Privileges Review

GCP now has a permission recommender
which will recommend least privilege based on actual usage. It can be used to determine minimum permissions in non-production environments. It should be reviewed periodically as part of scheduled reviews and maintenance.

Identity Providers

Okta is our corporate identity and authentication provider. Configuration of applications using Okta as a SAML provider is the preferred solution. It meets operation needs for security monitoring of activity and can be provisioned by IT Ops using the standard Access Request process. Applications designed to work as SAML Service Provider should be able to work with other identity providers if that changes in the future.

For applications requiring shell access for daily work, Okta ASA should be implemented for ssh authentication.

SECTION: security/product-security/_index.md

Aligned with GitLab's overarching information security strategy and its three-year plan, the Product Security Department (PSD) within the Security Division is responsible for crafting and directing a comprehensive vision to bolster the cybersecurity posture of the GitLab platform.

What is Product Security at GitLab?

At GitLab, product security encompasses a broad range of cybersecurity disciplines that enable product and engineering teams to design, develop, deploy, maintain, and refine GitLab's technologies securely. This goes beyond the conventional confines of security, covering everything from protecting developer workstations to ensuring the integrity of our production environments.

The Product Security Mission

Our mission is to set the standard for product security by fostering a culture of rapid innovation and secure product delivery. We are committed to leveraging the GitLab platform, embodying the pinnacle of internal usage ('dogfooding') practices. By maintaining close collaboration with product teams and contributing significant security features and capabilities to the GitLab codebase, we aim to enhance our operations and be a vital driver of the broader GitLab vision.

Multi-Year Product Security Mission

Our comprehensive, multi-year product security mission can be found in our internal handbook.

Product Security Risk Register

Our Product Security Risk Register process details can be consulted on this dedicated page.

Collaboration is Key

Success in product security is not confined to PSD or even the Security Division. It requires a concerted effort across the entire GitLab ecosystem. Collaboration is crucial, involving not just our security counterparts but the broader organization. Key disciplines and capabilities, from Security Operations to Site Reliability Engineering, while not directly under PSD's purview, are vital to our strategy's success.

Guiding Principles

- Business Enablement: PSD's role is to facilitate GitLab in achieving its business goals by ensuring product teams can operate both efficiently and securely, bolstering customer trust, and utilizing transparency as a strategic advantage. This includes providing insightful product feedback through extensive dogfooding.
- Empathy and Accessibility: Recognizing that the optimal security solution may not always align with business or customer needs, PSD prioritizes understanding and empathizing with these unique perspectives. This empathetic approach guides our security practices and engagements, aiming to align our methods with the preferences of our customers and internal teams.
- Pragmatism Over Perfection: Addressing current challenges quickly and effectively is preferred over waiting for a perfect solution. PSD focuses on delivering incremental, tangible value through rapid, short-cycle initiatives, aiming for partial solutions that immediately benefit our long-term goals.
- Design for Rapid Iteration: Our strategy and roadmaps are crafted to quickly identify and learn from suboptimal decisions by engaging with customers and stakeholders early and maintaining a tight feedback loop. This approach helps us adapt and refine our strategies and

approaches efficiently.
- Data-Driven Decision Making: Data drives our objectives, priorities, and actions, reducing the risk of failure or scope creep. Example useful metrics include root cause analyses of incidents (data within), threat modeling outcomes, and production readiness assessments, among others.
- Scalability and Repetition: PSD emphasizes scalable, repeatable processes over bespoke solutions, ensuring we can meet growing demands without proportional increases in resources.
- Decentralization and Empowerment: Acknowledging that product and engineering teams possess deep, specialized knowledge of their domains, PSD advocates for these teams to take ownership of security tasks like secure code reviews and threat modeling. This decentralization fosters a more integrated and effective security posture across GitLab.
- Integration with Reliability, Quality, Infrastructure, and Platform Engineering: PSD's mission to mitigate product security flaws is inherently tied to improving overall product quality and reliability. We aim to leverage and integrate with the practices of existing teams to enhance both security and product excellence.

Teams

The Product Security sub-department includes the following teams. Learn more about each by visiting their Handbook pages.

- Application Security
- Infrastructure Security
- Vulnerability Management
- Security Platforms and Architecture
- Data Security

SECTION: security/product-security/application-security/_index.md

<!-- markdownlint-disable MD052 -->
Last updated: May 27, 2025

Application Security Mission

The Product Application Security subdepartment works with GitLab engineers and product teams to anticipate and prevent the introduction of vulnerabilities during design and development, ensuring delivery of high quality software GitLab customers can trust. We also identify, assess, and respond to security vulnerabilities discovered in GitLab products and services that are reported through Coordinated Vulnerability Disclosure practices.

Value Proposition

The Application Security subdepartment provides operational application of DevSecOps engineering and methodology, as well as data insights and security consultation that enables GitLab engineers to easily deliver high quality secure products and services to customers, while maintaining feature capabilities and velocity to market.

Scope & Responsibilities

We organize our work into five pillars that emphasize Developer UX in the context of traditional DevSecOps programs. We call this the Secure Developer eXperience, or SDX.

- SDX: Learn: security training, governance, policy, documentation, and standards.
- SDX: Design: threat modeling, feature design guidance and consultation, and design reviews.
- SDX: Code: static analysis, software component analysis and supply chain security, use of approved tools and methodologies in development, deprecation of unsafe functions, etc.
- SDX: Verify: dynamic analysis testing, penetration testing, remediation of critical vulnerabilities, and final security reviews prior to release.
- SDX: Maintain: establishment of an incident response plan, managing Coordinated Vulnerability Disclosure, bug bounty program administration, and critical product security incident response release and post-release operations.

The Application Security sub-department includes two teams, the Secure Design & Development Team and the Product Security Incident Response Team (PSIRT).

Shared Accountabilities & Collaborations

The Application Security team partners with several other teams across the Security Division to deliver end-to-end security solutions that work for GitLab engineers. The following strategic security programs have multiple stakeholders across the Security Division and company.

Supply Chain Security

Application Security's accountability is shared by both SD&D and PSIRT. Additional Product Security teams involved in Supply Chain Security include Security Platforms & Architecture, Vulnerability Management, and Infrastructure Security.

Dogfooding

Application Security's accountability is to use GitLab security products in our work and be participants in providing actionable Customer Zero feedback through the Security Platforms & Architecture team, who is the Dogfooding DRI for Product Security.

Vulnerability Management

The Application Security Team's accountability is shared by both SD&D and PSIRT. The Vulnerability Management is DRI for Vuln Management tooling development and implementation.

Secure by design

The Secure Design and Development Team's accountability is feature focused, assessing threats through Threat Modeling and feature design reviews. (SDX: Design). The Security Platforms & Architecture team is DRI for Threat Modeling strategy company-wide, while AppSec is a critical stakeholder in this strategy.

Security Response

The Product Security Incident Response Team's accountability is to triage and technically assesses critical and exploitable vulnerabilities, determine company and customer risk, and

coordinate external communications regarding these issues. PSIRT has several partners across the company including:

- Security Operations is DRI for Incident Command and Threat Detection (IOCs, TTPs)
- Security Research is a key partner on exploitability and POC development
- PR and Communications
- Legal
- Delivery
- Customer Support

Out of Scope

- SBOM production
- Container Scanning
- Customer Escalations regarding security scanner findings
- Security Compliance

Contacting us

Team members can reach the AppSec team by:

- Finding your Stable Counterpart on the Product sections, stages, groups, and categories page
- Mentioning @gitlab-com/gl-security/product-security/appsec on GitLab
- Submit an issue in the AppSec Team repository
- Asking in sec-appsec or mentioning @appsec-team on Slack
- For cross team collaboration improvement opportunities, use this template for collaboration improvement opportunities

FY26 Primary Focus Areas

In FY26, our key focus areas are:

Organizational Upleveling:

- Establish Product Security Incident Response Team (PSIRT)
- Expand Security Design & Development team services at scale

Support Company and Division Priorities:

- Authorization & Authentication
- AI Security & Safety
- Supply Chain security
- Security Interlock

FY26 Metrics

Application Security is rebuilding our operational business health metrics in FY26. These metrics are in addition to Key Risk Indicators, project-level metrics, or sub-team specific metrics. For many of these, metrics instrumentation and reporting mechanisms are still forthcoming. As the team matures, these metrics will evolve and be shared on this page.

Useful resources for AppSec engineers

PTO

Team members that are taking PTO for 5 days or more must both discuss time off with their manager prior to scheduling to ensure visibility and adequate team operational coverage and create a PTO coverage issue to organize their coverage during their time off. The PTO coverage issue should:

- List any potential requests that could come to the team while on PTO
- The team member taking PTO should organize their work accordingly and ensure the PTO coverage issue contains the context required to handle the work
- Assign primary and secondary responsible team members

AppSec team members should add any important information related to the work they are covering for the person on PTO and AppSec manager(s) should add any important announcement to see upon their return.

Roles & Responsibilities

Please see the Application Security Job Family page.

Helpful Quicklinks

- The AppSec private group that contains other private subgroups and projects
- The appsec-lab group on Staging. This has an Ultimate license.
- Bug bounty council search
- Upcoming patch release
- GitLab Project Security dashboard
- Security issue board that tracks ongoing issues (hackerone and others)
- The latest releases
- Overview of a project member permissions
- The DevOps stages and their different groups. This page contains information on the development teams, their areas of focus, and their team members as well as the AppSec stable counterparts. It is used to assign issues to the stable counterparts.
- The product features listed by groups that own them
- List of merged security issues in gitlab-org. Note: It can include results from the security mirror gitlab-org/security/.
- Application Security KPIs & Other Metrics, including Embedded KPIs which can be filtered by section, stage, or group, please see this page.

The list above is not exhaustive and is subject to be modified as our processes keep evolving.

Stable Counterparts

Please see the Application Security Stable Counterparts page.

Application Security Reviews

Please see the Application Security Reviews page.

RCAs for Critical Vulnerabilities

Please see the Root Cause Analysis for Critical Vulnerabilities page

Application Security Engineer Runbooks

Please see the Application Security Engineer Runbooks page index

Meeting Recordings

The following recordings are available internally only:

- AppSec Sync
- AppSec Leadership Weekly

Backlog reviews

When necessary a backlog review can be initiated, please see the Vulnerability Management Page for more details.

GitLab Secure Tools coverage

As part of our dogfooding effort,
the Secure Tools are set up on many different GitLab projects (see our policies).
This list is too dynamic to be included in this page, and is now maintained in the GitLab AppSec Inventory.

Projects without the expected configurations can be found in the inventory violations list (internal link).

GitLab Inventory

Learn more about the GitLab AppSec Inventory.

Responding to customer scan review requests

Please see the Responding to customers security scanners review requests page.

Reproducible Vulnerabilities

Learn how to identify or remediate security issues using real examples with GitLab's Reproducible Vulnerabilities.

Reproducible Builds

Learn how GitLab is implementing Reproducible Builds for our build processes.

Milestone Planning

The GitLab Application Security team plans work based around Milestones, see this page for a

description of that process

Application Security Automation and Monitoring

Learn more about the automation initiatives that the Application Security team uses on the Application Security Automation and Monitoring page

Content Review and Updates

This charter will be reviewed quarterly to ensure alignment with company and divisional priorities, the GitLab Security product roadmap, and relevant business and operational changes. Updates may occur more frequently as business operations evolve.

Next scheduled review: June 30, 2025

SECTION: security/product-security/application-security/application-security-automation-monitoring.md

Monitoring

The Application Security team uses a number of automation initiatives to help secure GitLab. These are not all authored by the AppSec team but they're all useful to us. The points are listed in no specific order.

- Gem Checker monitors suspicious activity on RubyGems.org for gems that we use at GitLab
- sec-appsec-mr-alerts identifies MRs that modify dependencies used in our projects
- Public MR Confidential Issue Detector monitors for public merge requests that should have been opened in our security mirror
- Custom SAST rules detecting known-vulnerable code patterns that alert the AppSec team in the MR (related MR)
- untamper-my-lockfile included in CI to prevent lockfile tampering
- Package Hunter detects suspicious activity in dependencies at runtime (related runbook)
- GitLab Inventory monitors our projects and violations of security best practices and standards
- GitLab's own application security features are running in CI
- Tokinator monitors for leaked credentials
- AppSec Escalator which is a tool that...
  - monitors that security issues are labeled properly
  - sets appropriate due dates on security issues
  - escalates overdue issues
  - detects potentially sensitive files posted in public issues
- depSASTer runs SAST on the dependencies used by GitLab
- Maintainer Watcher monitors potentially compromisable dependency maintainer accounts
- depscore runs dependency review checks on new/updated depndencies in gitlab-org/gitlab project.

SECTION: security/product-security/application-security/appsec-async.md

Overview

As the Application Security team spans too many different time zones to have a reasonable schedule for a team-wide synchronous meeting we'll try to handle most discussions asynchronous.

The main problem to solve is knowing that other team members have had a chance to respond to a given issue.

needs-eyes Label

In order to point other AppSec team members we use the needs-eyes label under https://gitlab.com/gitlab-com/gl-security/product-security/appsec/

- Technical issues which need eyes should be create as meta-issues under gitlab-com/gl-security/product-security/appsec/appsec-reviews
- Non-technical should be created under gitlab-com/gl-security/product-security/appsec/appsec-team

Within the labeled issues any asynchronous discussion can take place. If a team member has read the issue but has no further input it should be marked acknowledged by a ·· emoji reaction.

When the team member which labeled the issue is happy with the results of the discussion the needs-eyes label can be removed.

Synchronous meetings

Within the needs-eyes labeled issues team member can decide to switch to synchronous communication and schedule a Zoom meeting in order to resolve questions more quickly.

If this happens the date/time and Zoom URL should be noted in the issue to give other team member the chance to join in. Additionally the meeting should be recorded and made available to the whole AppSec team.

SECTION: security/product-security/application-security/appsec-dogfooding-feature-requests.md

Overview

This page describes the usage of a label to indicate a specific issue or epic is a priority for the Application Security (AppSec) team. This label helps in categorizing, tracking, and sharing product features requests that would benefit the AppSec team as part of dogfooding the GitLab product.

~"Internal Request::AppSec Team" Label

- Link to the list of issues with this label on gitlab-org.

- Link to the list of issues with this label on gitlab-com.

Usage

Apply the ~"Internal Request::AppSec Team" label when the AppSec team requests new security features or improvements to be built into GitLab, the product.

Important Note

This label is not applicable for vulnerabilities. Issues related to vulnerabilities should use the ~bug::vulnerability label.

SECTION: security/product-security/application-security/appsec-reviews.md

<!-- markdownlint-disable MD052 -->

This page details the application security review process for appsec engineers. The purpose of application security reviews are to proactively discover and mitigate vulnerabilities in GitLab developed or deployed applications in order to reduce risk and ultimately help make the company's mission successful.

An application security review may include any or all of the following stages:

- Threat modeling
- In-depth code review
- Dynamic testing

The results of each stage will inform the review done in the next stage. Ideally, all new features would receive some threat modeling, with the latter two stages being performed based on the risk profile. Features already in development or production can receive an appsec review as well. The testing done is dependent on the circumstances.

What does an application security review mean for the team owning the feature?

A security review conducted by the application security team is non-blocking. This means that the team owning the feature should continue with their development plan, and the expected time investment should be limited to the time necessary to answer questions asynchronously.

Security issues found, if any, will be triaged following the standard process. Application security reviews allow us to discover vulnerabilities that exist in GitLab before they're discovered by a third party and, if the review is done for new features, we might catch the vulnerabilities even before they make it into a release. It reduces risk, gives us a better
understanding of the threat model of the given feature, and allows us to proactively mitigate vulnerabilities.

Roles & Responsibilities

Teams

The team owning the feature should proactively involve the Stable Counterpart in Epics, Issues, and/or MRs which might require a review or their attention. This is primarily the responsibility of the team's Engineering Manager(s) and the Engineer(s) working on the Issue / MR. Ideal trigger points (in order of preference) are when the Epic/Issue/MR is created/updated, when an engineering proposal is updated, or when an engineer is working on the MR.

Teams should have a bias towards involving the Stable Counterpart, to prevent potential security-sensitive changes from being overlooked.

Stable Counterparts

One of the goals of the stable counterpart is to help identify features that need security review in the area to which they are assigned. Each week Stable Counterparts should review Issue boards and recorded weekly team meetings as part of this.

An Application Security team member is on rotation each week to triage Application Security Reviews.

What Should Be Reviewed?

The application security review queue is a priority queue of application features for security review. The priority can range from priority::1 (Critical) to priority::4 (Low/Backlog).

Some guidelines for which features should be added to the queue are:

- All new major features
- Features that have had repeat vulnerabilities
- Features related to authorization or authentication
- Features that handle Red or Orange data
- Features that work with cryptography or other data protection solutions
- Features which touch on topics mentioned in the secure coding guidelines

The idea is to capture features determined to be higher risk for vulnerabilities. It is quite probable that all features, especially priority::4 issues, will not get a full review, but by capturing those that are at higher risk, we can track additional statistics. For example, how many related vulnerabilities were reported in the bug bounty program. This data can help us to help iterate on priority.

Single Issue / MR Pings

Single Issue/MR pings that can be completed by the engineer on triage rotation do not need a separate issue. This process is primarily for tracking features

over time. With that in mind, if a ping will need additional review, an issue should be created.

Adding Features to the Queue / Requesting a Security Review

Separate issues will be used to track the appsec review process, as this process could outlive the original issue/merge request.

The process is the same for appsec engineers adding something to the backlog or for team members requesting a review for a GitLab feature:

1. Create an issue in the Appsec Reviews issue tracker using the Appsec Review template
    1. Set the title to a unique name for the feature
1. Follow the description in the template

Assigning Priority

Every issue should have a priority assigned to help team members plan testing. It is up to the application security engineer creating the issue to determine priority based on the data available to them. If you are not sure of the appropriate priority, be conservative and assign a higher priority. It can always be adjusted given feedback from other team members.

Guidelines for Priority (Not comprehensive, please build upon)

Priority	Criteria
priority::1	Red data, AuthN/AuthZ, Crypto, Single severity::1, Repeat severity::2 vulns
priority::2	Orange data, Single severity::2 vulns
priority::3	Yellow data
priority::4	Only standard secure practices necessary

Including Threat Modeling in the review

When threat modeling should be done during the review add the threat model::needed label to the original issue or epic and the appsec review issue. That way we can track the adoption of threat modeling throughout GitLab. When the threat modeling step is done the threat model::done label should be added to all involved issues and epics. The process for threat modeling is further defined in its own handbook page.

Quantifying interactions

The engineering team has created multiple labels with the purpose of quantifying interactions done by stable counterparts and tracking the status of these interactions. Stable counterparts should add the right label depending on the status of the interaction following the conditions below:

- ~sec-planning::in-progress: The issue or MR is under review.
- ~sec-planning::pending-followup: Stable counterpart expects to follow up on the review.
- ~sec-planning::complete: Review finished with comments.
- ~sec-planning::no-action: Review completed and no action required.

Internal Application Security Reviews

For systems built (or significantly modified) by Departments that house customer and other sensitive data, the Security Team should perform applicable application security reviews to ensure the systems are hardened. Security reviews aim to help reduce vulnerabilities and to create a more secure product.

When to request a security review?

This short questionnaire below should help you in quickly deciding if you should engage the application security team:

If the change is doing one or more of the following:

1. Processing, storing, or transferring any kind of RED or ORANGE data
1. If your changes have a goal which requires a cryptographic function such as: confidentiality, integrity, authentication, or non-repudiation, it should be reviewed by the application security team.
1. Deployment of a customer facing application into a new environment
1. Changes to an existing security control
1. Modification of any pipeline security checks or scans
1. A new authentication mechanism
1. Adding code that touches the authentication model, tokens or sessions
1. Dealing with user supplied data
1. Touching cryptography functions, see the GitLab Cryptography Standard for more details
1. Touching the permission model
1. Implement new security controls (i.e. new library for a specific protection, HTTP header, ...)
1. Exposing a new API endpoint, or modifying an existing one
1. Introducing new database queries
1. Using regex to :

   - validate user supplied data
   - make decisions related to authorisation and authentication

1. A new feature that can manipulate or display sensitive data (i.e PII), see our Data Classification Standard for more details
1. Persisting sensitive data such as tokens, crypto keys, credentials, PII in temp storages/files/DB, manipulating or displaying sensitive data (i.e PII), see our Data Classification Standard for more details

You should engage @gitlab-com/gl-security/product-security/appsec.

How to request a security review?

There are two ways to request a security review depending on how significant the changes are. It is divided between individual merge requests and larger scale initiatives.

Individual merge requests or issues

Loop in the application security team by /cc @gitlab-com/gl-security/product-security/appsec in your merge request or issue.

These reviews are intended to be faster, more lightweight, and have a lower barrier of entry.

Larger scale initiatives

To get started, create an issue in the internal application security reviews repository using the Appsec Review template. The complete process can be found at here.

Some use cases of this are for epics, milestones, reviewing for a common security weakness in the entire codebase, or larger features.

Is security approval required to progress?

No, code changes do not require security approval to progress. Non-blocking reviews enables the freedom for our code to keep shipping fast, and it closer aligns with our values of iteration and efficiency. They operate more as guardrails instead of a gate.

What should I provide when requesting a security review?

To help speed up a review, it's recommended to provide any or all of the following:

- The background and context of the changes being made.
- Any documentation or diagrams which help provide a clear understanding its purpose and use cases.
- The type of data it's processing or storing.
- The security requirements for the data.
- Your security concerns and a worst case scenario that could happen.
- A test environment.

What does the security process look like?

The current process for larger scale internal application security reviews be found here

My changes have been reviewed by security, so is my project now secure?

Security reviews are not proof or certification that the code changes are secure. They are best effort, and additional vulnerabilities may exist after a review.

It's important to note here that application security reviews are not a one-and-done, but can be ongoing as the application under review evolves.

Using third party libraries ?

If you are using third party libraries make sure that:

1. You use the latest stable and available version
1. Your team has the ability to support and upgrade this library as security patches are published
1. The maintainer has a security policy

MR Review guidelines

When concluding a review of a Merge Request make sure you document what you covered, and what your conclusions on the reviewed items were.

Having a summary of your steps taken, concerns and coverage helps collaboration with other reviewers.

1. Coverage: Clearly outline what aspects of the code you have reviewed. This may include:
   - Specific files or modules examined
   - Functionality changes assessed
   - Security implications considered

2. Steps Taken: Provide a brief overview of your review process, such as:
   - Code reading and analysis
   - Local testing or code execution
   - Use of any automated tools or linters
   - Cross-referencing with related issues or documentation

3. Concerns and Observations: Document any issues, potential problems, or areas for improvement you've identified during the review. This could include:
   - Security vulnerabilities
   - Code quality issues
   - Potential bugs or edge cases
   - Suggestions for optimization

4. Conclusions: Summarize your overall assessment of the reviewed items, including:
   - Whether the changes meet the intended requirements
   - Any blockers or critical issues that need addressing
   - Recommendations for further actions or improvements

By providing a comprehensive summary of your review process, concerns, and coverage, you facilitate:

- Improved collaboration with other reviewers
- Easier follow-up on identified issues
- A clear record of the review process for future reference
- More efficient resolution of any concerns or questions raised during the review

A well-documented MR review not only helps in the immediate code review process but also contributes to the overall quality and maintainability of the project in the long run.

SECTION: security/product-security/application-security/inventory.md

GitLab Application Security Inventory

Securing GitLab means building a security program at scale. The number of changes in the codebase is constantly increasing, along with the number of side projects.
Keeping track of all these moving parts can not rely only upon our current understanding and vision of the GitLab software architecture.
Automation is a key aspect of our work, and GitLab is no exception.

The AppSec Inventory is a private GitLab project to identify and track all projects, components, and dependencies important to us.
The project is available at https://gitlab.com/gitlab-com/gl-security/product-security/inventory
to GitLab team members. The Inventory is built using this CLI tool.

Not all projects are important, and we certainly don't want to monitor projects that are POCs, demos, or tests.
That's why we need to categorize the projects created by GitLab team members, understand their nature, and make decisions at scale.

Categories

To quickly identify the purpose and characteristics of a project, a strict categorization is necessary.
The following categories can be used to decorate the projects we want to monitor.

Categories	Description
product	Part of GitLab's software supply chain (i.e. used to build, package, release, deploy GitLab, or used as part of the product)
library	A library, package source, component (not necessarily a product one)
deploy	Used to deploy GitLab.com
website	Deployed to a website (URL will be required)
api/service	
green/yellow/orange/reddata	Data classification standard
3rdparty	Interaction with 3rd parties
demo/test/poc	
temporary	Temporary projects (should be removed at some point)
internal	Available for GitLab team members only
external	User facing
usepat	Personal Access Token being used
markedfordeletion	Project should be removed
keepprivate	Should remain private indefinitely
docs	Used to generate documentation
tooling	Engineering tooling
container	A Docker image is built
fork	Fork of another project (on gitlab.com or somewhere else)
secretsmonitoring	Monitor secrets in groups (see this confidential issue)

| securitypolicyproject | GitLab security policy projects |

Policies

We apply several policies depending on the categories defined above. These policies, which include security requirements, are available here and in our (internal only) inventory.

They are used are controls for our GitLab Projects Baseline Requirements.

How to categorize projects

The inventory relies on a folder tree structure, used as a database, in a data/ folder.
Leaves are folders and can be groups or projects, and they're identified by specific files (project.json for projects, group.json for groups).
These files are created automatically when syncing the Inventory.

The tree structure reflects the organization of groups and projects in a GitLab instance, in our case: https://gitlab.com.
For example, the GitLab project will be located under data/gitlab-org/gitlab/ in the Inventory.

Projects can be categorized by creating a properties.yml file in their folder. This file can contain a categories array, with the categories of the project.

For example, to add the product and library categories:

yaml
categories:
  - product
  - library

Subgroups can be ignored (skipped during synchronization) by adding an ignore file into their folder.

Learn more with the GitLab Inventory Builder Documentation, and this example inventory.

How to add or update your GitLab Project

1. Note the namespace of your project.
1. Visit <https://gitlab.com/gitlab-com/gl-security/product-security/inventory/-/tree/main/data/>
1. Navigate the folder structure to find your project's existing properties.yml file.
    1. If your project does not exist, create a file at data/your-namespace/your-project/properties.yml.
    1. Projects created in GitLab's namespaces are added automatically on a weekly basis.
1. Open a Merge Request that adds or updates the categories. Remember to "say why, not just what".

Websites

Along with the categorization of the projects, the Inventory is also used to link websites we deploy with their projects. The properties.yml file can contain a urls array to list all the URLs (starting with https://) of a project. These URLs are used to validate the SSL configuration of the servers, and insecure findings are reported.

For example, to add the GitLab website URL:

yaml
urls:
  - https://gitlab.com

Weekly triage process

A synchronization pipeline runs every week, on Monday mornings. If successful, it will generate a Merge Request to review the changes.

The review aims to:

1. Categorize newly created projects: Add a properties.yml file in the project folder to specify its categories. Ask the project owner in doubt.
1. Ignore newly created groups we don't want to track: Add an ignore file if the group should be ignored. Delete its subgroups and projects in the review Merge Request.
1. Review projects and groups updates: Review project.json and group.json for changes, especially the visibility (public/private).

The Merge Request will report a test coverage, corresponding to the ratio of projects categorized. Ideally, these review Merge Requests keep the same coverage, which means the new projects are categorized before getting merged.

SECTION: security/product-security/application-security/milestone-planning.md

Milestone Planning

The Application Security team plans its work on a cadence based around GitLab Product Milestones. This enables us to be intentional about the work we do, while also providing insights into our capacity, velocity, and current projects.

This handbook page describes the planning process that we use to determine what work will be completed for each Milestone.

Milestone Planning Issue

For each Milestone, a Milestone Planning issue is created in the Application Security team repository. The purpose of this issue is to:

- Identify potential work to perform: rotations, KR work, critical project work, and other efforts

- Identify refinement gaps and address them
- Determine what work we're committing to for the upcoming Milestone
- Set and communicate priority for the work we've decided to take on

This issue is the single source of truth for all planning related discussions and decisions related to the upcoming Milestone.

Milestone Planning Process

1. On the first of the month, an Application Security manager will create an issue from the Milestone Planning issue template
1. An Application Security manager will be responsible for completing the checklist items in the Planning Checklist section of the Milestone Planning issue
1. Application Security team members will add any work being carried over from the previous Milestone into the Milestone Work table
1. The Application Security team will add potential work items to the Parking Lot section, with a brief explanation of why it would be good to include in the Milestone
    1. Team members can add discussion threads to discuss potential work to pull into the Milestone
    1. Both individual team members and Application Security managers can add items to this list
1. The Application Security team will work together to add new items to the Milestone Work table
    1. Each item being added must be refined before it can be formally committed to
    1. The team member likely to take on the work should review and agree with the Weight, if it wasn't them who refined the issue
    1. Once we have refined and committed to the work, the relevant issue needs to be updated with the Milestone and Assignee(s)
1. The Milestone Planning issue should be finalized at least 3 business days before the Milestone Start Date
    1. The Application Security Manager will use threads in the Milestone Planning issue to work with each Application Security team member to finalize their workload
    1. Once finalized, the Milestone Planning issue should be closed

Milestone Planning responsibilities

Application Security team members are responsible for:

- Evaluating and communicating their capacity for the Milestone (based on PTO, rotation assignments, and other factors)
- Adding potential work items to the Parking Lot and being involved in discussions around what work we should pull into the Milestone
- Verifying upcoming rotations they are assigned to have an issue in the Milestone
- Collaborating with Application Security managers to finalize the set of work being committed to for the Milestone

Application Security managers are responsible for:

- Creating, updating, and maintaining the Milestone Planning issue
- Creating and properly labeling the upcoming Rotation issues

- Collaborating with Application Security team members to discuss potential work, identify refinement gaps, and assemble the Milestone Work table
- Coordinating the finalization of the Milestone Planning issue

Issues and Labels

Application Security team members are responsible for keeping issues and labels up-to-date over the course of the Milestone.

Any issue being worked on by an Application Security team member must include:

- The Application Security Team label
- The appropriate AppSecWorkflow:: label
- The appropriate Milestone
- The appropriate Priority labels

Updating issues health

DRIs are responsible for providing weekly updates at the end of the week on their assigned topics using the following format.

```
md
 What's happened since last update:

  \[Bullet points of progress\]

 What's next:

  \[Bullet points of upcoming work\]

 Blockers:

  \[Any blockers or dependencies\]

 Overall Status/Confidence:

  :greencircle: On Track / :yellowcircle: Needs Attention / :redcircle: At Risk
  \[Brief explanation of status\]
```

These reports provide critical visibility into progress, plans, and potential issues, allowing leadership to make informed decisions and offer timely support when needed.

Updating the issue milestone is required:

- On a weekly basis, at the end of the week.
- Whenever the DRI knows they won't be able to finish it for the end of the milestone

Workflow Labels

Label	Purpose
AppSecWorkflow::planned	Indicates that work has been triaged, scoped, and is ready to be worked on in the assigned milestone
AppSecWorkflow::in-progress	Indicates the issue is actively being worked on, or the rotation is in progress
AppSecWorkflow::complete	Indicates the work is done, or the rotation has finished

Priority Labels

The priority classification labels helps ICs understand what is the priority for leadership.

The label assignment can be done by leadership (AppSec or at higher levels), or by the team members themselves. When team members are not sure on a particular priority, they can consult leadership for confirmation.

Label	Description
AppSecPriority::1	Top priority work that must be completed for the end of the planned milestone.
AppSecPriority::2	Work priority that is important and is prioritized as soon as all AppSecPriority::1 work is completed. AppSecPriority::2 work will become AppSecPriority::1 on the next milestone.
AppSecPriority::3	Work priority that is less important and is prioritized as soon as all AppSecPriority::2 work is completed. AppSecPriority::3 will be evaluated during Milestones Planning Sessions and may become AppSecPriority::2 for the next milestone.

Rotations

HackerOne and Triage rotation issues are created through the rotation management tool

Milestone Planning Refinement Guidelines

- Is the problem clearly defined or is more followup/data needed?
- Is the scope too large to be completed within the milestone? Does the issue need to be broken down into smaller ones or promoted to an epic instead?
- For projects and net-new initiatives, is the scope and Definition of Done clear and measurable? Is it clear what's expected?
- Does it have at least one DRI assigned and are they aware?
- Are there dependencies? If so, document them.
- Are there other stakeholders and are they looped in and aware?
- Is the correct AppSecWorkType:: label set?
- Is the AppSecWeight:: label set?
- Does it have the Application Security Team label?
- Across the whole milestone, is the total operational + project weight achievable?

When issue is fully refined, please set the AppSecWorkflow:planned label, indicating it's ready to be worked on in the assigned milestone.

Unplanned work

Sometimes high-priority and/or urgent work comes up after a milestone starts. When an unplanned issue is added after the milestone began:

- Document why the work needs to be prioritized in the issue
- Apply the Unplanned label
- If the unplanned work is large enough to displace other planned issues, inform the applicable stakeholders so they are aware that their issue is being delayed

Missed milestones

Work planned for a milestone may not be fully finished due to time constraints or planned work being too ambitious. When this happens, attach the missed::X.Y label.

Backlog

Issues that:

- Have unfinished work
- Are ideas from team members but not yet prioritized
- Are not planned in any milestone

Should have the milestone set to AppSec Backlog.

SECTION: security/product-security/application-security/reproducible-builds.md

What are Reproducible Builds?

Reproducible Builds (RB) is a methodology for building software that allows the end user to verify that the build published on any given release is sufficiently similar[^1] or the same as the build that is deployed locally.

The RB methodology is composed of three major tenants:

- Hermetic: All build inputs are specified in an unambiguous way. Each item that goes into a build is given a fully resolved version number and/or cryptographic hash. Hermeticity is verified as part of the build process.
- Reproducible: When you run the build process, you should get the same bit-by-bit output as the developer. All sources of nondeterminism and variance should be removed and all information necessary to reproduce the build should be provided up front in a buildinfo or similar lock file.
- Verifiable: Mechanisms should be employed to cryptographically verify the end binary to the source it was built from in a trustworthy manner.

Why are Reproducible Builds Important?

RB allows for end users to have confidence in the software supply chain for a built component. It hardens the software build process from threats like backdoors and other in-transit vulnerabilities that can arise from distribution mirrors, third party hosting and so on. It

also assists in debugging. Builds with problems that have a low RB similarity score can be more easily triaged.

What are the Technical Requirements for Hermeticity?

Builds need to implement binary provenance (BP). Binary provenance is another way of saying that the build system should have descriptions of exactly how a given binary artifact is built. This includes the inputs, the transformation and the entity that performed the build. The BP is a file that includes the following fields:

- Authenticity Attestation: describes system that produced the BP and includes cryptographic signatures to provide attestation as to the authenticity for the file
- Expected Outputs: describes all expected outputs of the build process as well as provides a cryptographic hash for each artifact.
- Expected Inputs: describes everything included in the build. This field allows the verifier to correlate the source to the artifact
  - Sources: within the expected inputs field. This might look like "git commit 291e...494d" from https://gitlab.com/gitlab-org/gitlab" or even "gitlab-ee-17.0.1" with SHA256 hash "75a4...9982"
  - Dependencies: Every implicit dependency needs to be linked in a format above. Each input affects the integrity of the build.
- Commands: each command used to initiate the build needs to be outlined in such a way as to allow for automated analysis
  - Example: {"gitlab-omnibus": {"command": "build", "target": "//main:helloworld"}}"
- Environment: any information such as architecture details, environment variables, tool/compiler versions, etc.
- Versioning: Build timestamps

Note that other fields may be beneficial and the list above is not comprehensive. Each BP should be modified to fit the appropriate threat landscape. Builds that require higher fidelity should include additional fields to aid in verification.

What are the Technical Requirements for Reproducibility?

As mentioned above, different Reproducible Builds are going to have different required similarity scores. It's not always feasible to enforce 100% reproducibility as some nondeterministic factors may be unable to be mitigated. In order to meet the criteria for reproducibility, builds should be:

- Byte-by-byte verifiable: this typically takes the form of a cryptographic hash that is able to be compared against an authenticated source
- Automatable: build hashes of source inputs, expected outputs, attestations, and so on should be provided in a format that is easily able to be verified by automation

[^1]: Sufficient similarity highlights that 100% verifiability is not always feasible. Variances across compiler versions, OS hardware, timestamps, ordering of files on a filesystem, and other sources of nondeterminism will degrade similarity metrics. Sufficient similarity goals are to be set by the source code maintainers.

SECTION: security/product-security/application-security/reproducible-vulnerabilities.md

Introduction

GitLab is tra